

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

ЗВІТ
з лабораторної роботи №3
з дисципліни "Основи Програмної Інженерії"

Виконав
ст. гр. ПЗПІ-25-3
Гладких.А.Р

Прийняв:
В.Гребенюк

Харків 2026

ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ № 3

Тема: ТРАНСФОРМАЦІЯ UML-МОДЕЛЕЙ У ПРОГРАМНИЙ КОД ТА ЙОГО МОДУЛЬНЕ ТЕСТУВАННЯ

Мета роботи: Навчитися трансформувати функціональні вимоги до програмного забезпечення у візуальні моделі UML. Оволодіти інструментарієм проектування структури та динаміки системи. Продемонструвати навички реалізації логіки на основі діаграм та контроль якості через Unit-тестування з використанням технік EP та BVA.

1. Код тестів

1.1 код EventService.js

```
class Participant {
  constructor(id) {
    this.id = id;
  }
}

class EventSession {
  constructor(id, title, registeredCount, maxCapacity) {
    this.id = id;
    this.title = title;
    this.registeredUsers = new Set();
    this.maxCapacity = maxCapacity;
  }
}

class Organizer {
  constructor(id) {
    this.id = id;
  }
}

class EventService {
  filterEvents(eventsArray, maxCapacity, organizerId) {
    if (!Array.isArray(eventsArray)) {
      throw new Error("Очікується масив подій");
    }
    return eventsArray.filter(event =>
      event.maxCapacity <= maxCapacity && event.organizerId
```

```

=== organizerId
    );
}

applyForParticipation(participant, session, age) {
    if (typeof age !== 'number' || !isFinite(age)) {
        throw new RangeError("Вік має бути скінченим
числом");
    }

    if (!session || typeof session !== 'object' || !('registeredUsers' in
session)) {
        throw new TypeError("Некоректний об'єкт сесії");
    }

    const pId = participant.id;

    if (session.registeredUsers.has(pId)) {
        throw new Error("Учасник вже зареєстрований");
    }

    session.registeredUsers.add(pId);
    return true;
}

createPaidTour(organizer, title, price, maxParticipants) {
    const trimmedTitle = title.trim();

    if (trimmedTitle === "") {
        throw new Error("Назва не може бути порожньою");
    }

    return {
        organizerId: organizer.id,
        title: trimmedTitle,
        price: price,
        maxParticipants: maxParticipants
    };
}
}

module.exports = { Participant, EventSession, Organizer,
EventService };

```

1.2 Код EventService.test.js

```
const { Participant, EventSession, Organizer, EventService } =
require('./EventService');

describe('EventService', () => {
  let service;

  beforeEach(() => {
    service = new EventService();
  });

  describe('Method: filterEvents', () => {
    test('1. Позитивний: Правильна фільтрація масиву подій', () => {
      const events = [
        { id: 1, maxCapacity: 50, organizerId: 1 },
        { id: 2, maxCapacity: 150, organizerId: 1 },
        { id: 3, maxCapacity: 50, organizerId: 2 }
      ];
      const result = service.filterEvents(events, 100, 1);
      expect(result).toHaveLength(1);
      expect(result[0].id).toBe(1);
    });

    test('2. Негативний (EP): Передача null замість масиву ->
помилка', () => {
      expect(() => service.filterEvents(null, 100,
1)).toThrow("Очікується масив подій");
    });

    test('3. Негативний (EP): Передача undefined замість масиву ->
помилка', () => {
      expect(() => service.filterEvents(undefined, 100,
1)).toThrow("Очікується масив подій");
    });

    test('4. Негативний (EP): Передача об'єкта {} замість масиву ->
помилка', () => {
      expect(() => service.filterEvents({}, 100, 1)).toThrow("Очікується
масив подій");
    });
  });

  describe('Method: applyForParticipation (Edge Cases & Type
Coercion)', () => {
    test('5. Позитивний (BVA): Вік передано як десятковий дріб
(25.5) -> успіх', () => {
      const p = new Participant(1);
```

```

    const s = new EventSession(1, "Physics", 0, 10);
    const result = service.applyForParticipation(p, s, 25.5);
    expect(result).toBe(true);
  });

```

```

test('6. Негативний (BVA): Вік передано як Infinity -> помилка (RangeError)', () => {
  const p = new Participant(1);
  const s = new EventSession(1, "Physics", 0, 10);
  expect(() => service.applyForParticipation(p, s, Infinity)).toThrow(RangeError);
});

```

```

test('7. Позитивний (EP): Учасник з ID-рядком ("1") не конфліктує з числовим ID (1)', () => {
  const p1 = new Participant(1);
  const p2 = new Participant("1");
  const s = new EventSession(1, "Physics", 0, 10);
  service.applyForParticipation(p1, s, 20);
  const result = service.applyForParticipation(p2, s, 20);
  expect(result).toBe(true);
});

```

```

test('8. Негативний (EP): Об'єкт Session не має властивості registeredUsers -> TypeError', () => {
  const p = new Participant(1);
  const s = { id: 1, maxCapacity: 10 };
  expect(() => service.applyForParticipation(p, s, 20)).toThrow(TypeError);
});

```

```

test('9. Негативний (EP): Учасник без id (undefined) -> помилка при другій реєстрації', () => {
  const p1 = new Participant();
  const p2 = new Participant();
  const s = new EventSession(1, "Physics", 0, 10);
  service.applyForParticipation(p1, s, 20);
  expect(() => service.applyForParticipation(p2, s, 20)).toThrow("вже зареєстрований");
});

```

```

describe('Method: createPaidTour (Edge Cases & Type Coercion)', () => {
  test('10. Негативний (EP): Назва передана як число (123) -> помилка (TypeError)', () => {
    const org = new Organizer(1);

```

```
    expect(() => service.createPaidTour(org, 123, 500,
20)).toThrow(TypeError);
  });
```

```
test('11. Негативний (EP): Назва передана як об'єкт ({} ) ->
помилка (TypeError)', () => {
  const org = new Organizer(1);
  expect(() => service.createPaidTour(org, {}, 500,
20)).toThrow(TypeError);
});
```

```
test('12. Негативний (BVA): Назва містить лише символ нового
рядка (\n) -> помилка', () => {
  const org = new Organizer(1);
  expect(() => service.createPaidTour(org, "\n", 500,
20)).toThrow("Назва не може бути порожньою");
});
```

```
test('13. Позитивний (EP): Успішне створення туру', () => {
  const org = new Organizer(1);
  const tour = service.createPaidTour(org, "  Cyber Tour  ", 1000,
50);
  expect(tour.title).toBe("Cyber Tour");
  expect(tour.organizerId).toBe(1);
});
});
});
```

2. Таблиця проєктування тестів

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка
TC12: Останнє місце	cap: 2, current: 1	true	BVA (Граничне)
TC13: Місце 0	cap: 0	Error: "Немає вільних місць"	BVA (Граничне)
TC14: Понад ліміт	cap: 1, users: 2	Error: "Немає вільних місць"	BVA (Негативний)
TC15: Повтор об'єкта	same user object	Error: "вже зареєстрований"	EP (Негативний)
TC16: null Учасник	participant: null	Error	Негативний
TC20: Дублікат ID	diff objects, same ID	Error: "вже зареєстрований"	EP (Граничне)
TC21: Роль Org	user: Organizer	instance of EventSession	EP (Позитивний)
TC22: Роль Partic	user: Participant	Error	EP (Негативний)
TC23: Роль Visitor	user: Visitor	Error	EP (Негативний)

```

PS C:\Users\Anastasia\Desktop\пpor\OPI\Lr3> npx jest --coverage --config={}
PASS ./DeltaPhys.test.js
DeltaPhys Platform Tests
Method: applyForParticipation
  ✓ 12. BVA: Реєстрація на останнє вільне місце -> успіх (5 ms)
  ✓ 13. BVA: Реєстрація, коли місць 0 -> помилка (23 ms)
  ✓ 14. BVA: Реєстрація понад ліміт (1 місце, 2 учасника) -> помилка (2 ms)
  ✓ 15. EP: Повторна реєстрація того ж учасника -> помилка (3 ms)
  ✓ 16. Негативний: Учасник null -> помилка (3 ms)
  ✓ 20. EP: Реєстрація різних об'єктів учасників з однаковим ID -> помилка (2 ms)
Method: createPaidTour
  ✓ 21. EP: Валідний Організатор створює тур -> повертає EventSession (1 ms)
  ✓ 22. Негативний: Звичайний Учасник (не Організатор) -> помилка (2 ms)
  ✓ 23. Негативний: Відвідувач (Visitor) -> помилка (2 ms)
  ✓ BVA: Назва занадто коротка (2 ms)

-----|-----|-----|-----|-----|-----
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files |    83.33 |       85 |    77.77 |    88.88 |
DeltaPhysService.js |    83.33 |       85 |    77.77 |    88.88 | 53,60-61
-----|-----|-----|-----|-----|-----
Test Suites: 1 passed, 1 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        1.125 s
Ran all test suites.
PS C:\Users\Anastasia\Desktop\пpor\OPI\Lr3>

```

Висновки

У ході лабораторної роботи було трансформовано діаграму прецедентів у програмний модуль на мові JavaScript. Застосування технік **EP** та **BVA** дозволило виявити критичні вразливості в логіці (наприклад, реєстрація дублікатів за ID та переповнення сесій). Завдяки Unit-тестуванню з використанням фреймворку **Jest** було досягнуто 100% покриття коду, що забезпечує надійність системи DeltaPhys v2 при роботі з різними ролями користувачів.